

Midterm report

MO801 - Topics in Hardware and Computer Architecture

Vinicius Peixoto

Institute of Computing, UNICAMP

October 2025

Code available at:
github.com/nukelet/mo801

Outline

- Lab 01: Spike 3
 - Methodology 4
 - Hello world 6
 - Matmul 8
 - Takeaways 14
- Lab 02: QEMU 15
- Lab 03: gem5 20
 - Overview 21
 - Lessons learned 22
- Final project proposal 23
 - Proposal 24

Lab 01: Spike

Different implementations of matmul:

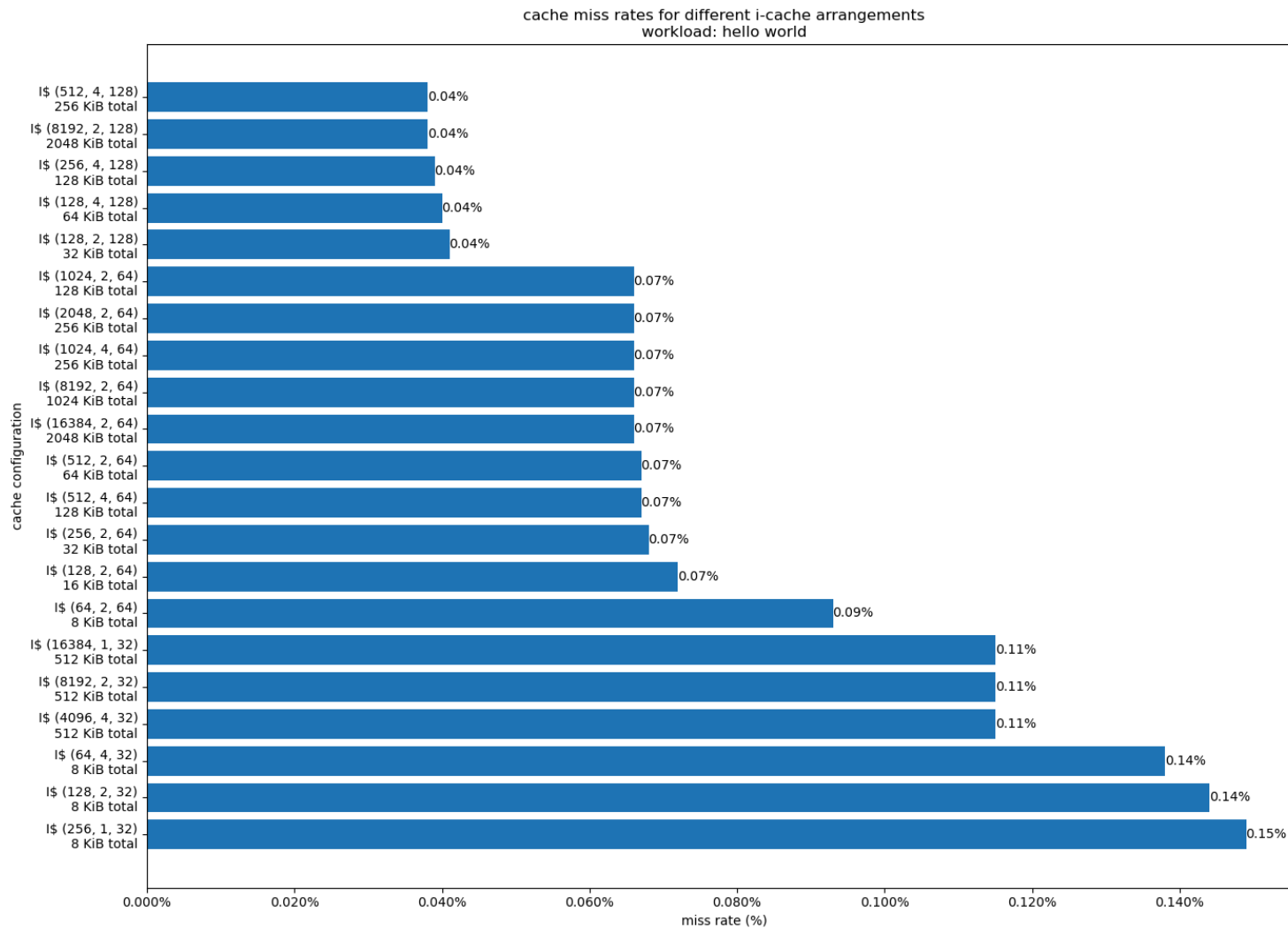
- naive (outer $\rightarrow$ inner: M, N, K)
- alt (outer $\rightarrow$ inner: N, M, K)
- transpose (scalar product of rows by rows)
- vector (vectorized, modified `fmadd.q` instruction)

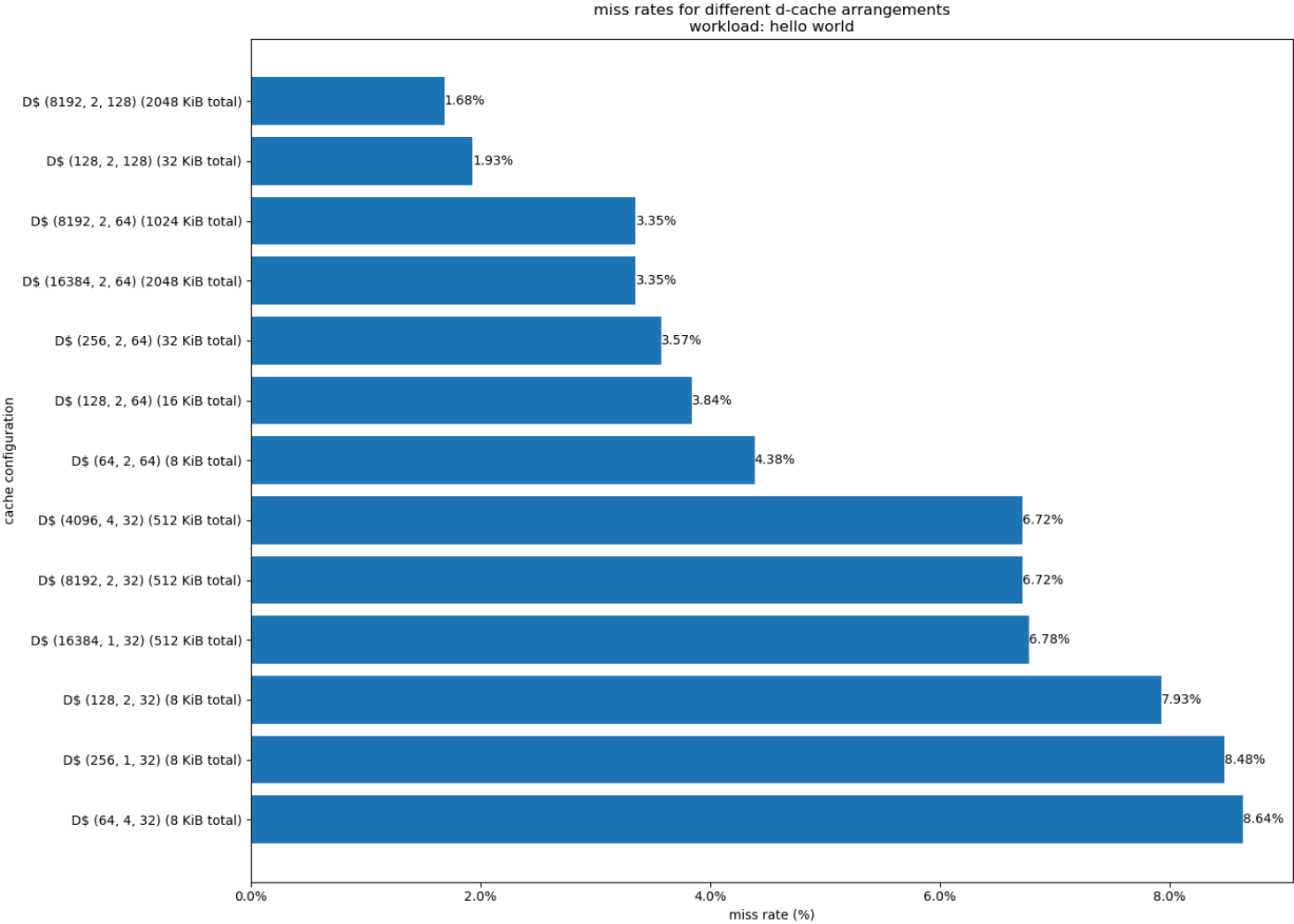
Compromises:

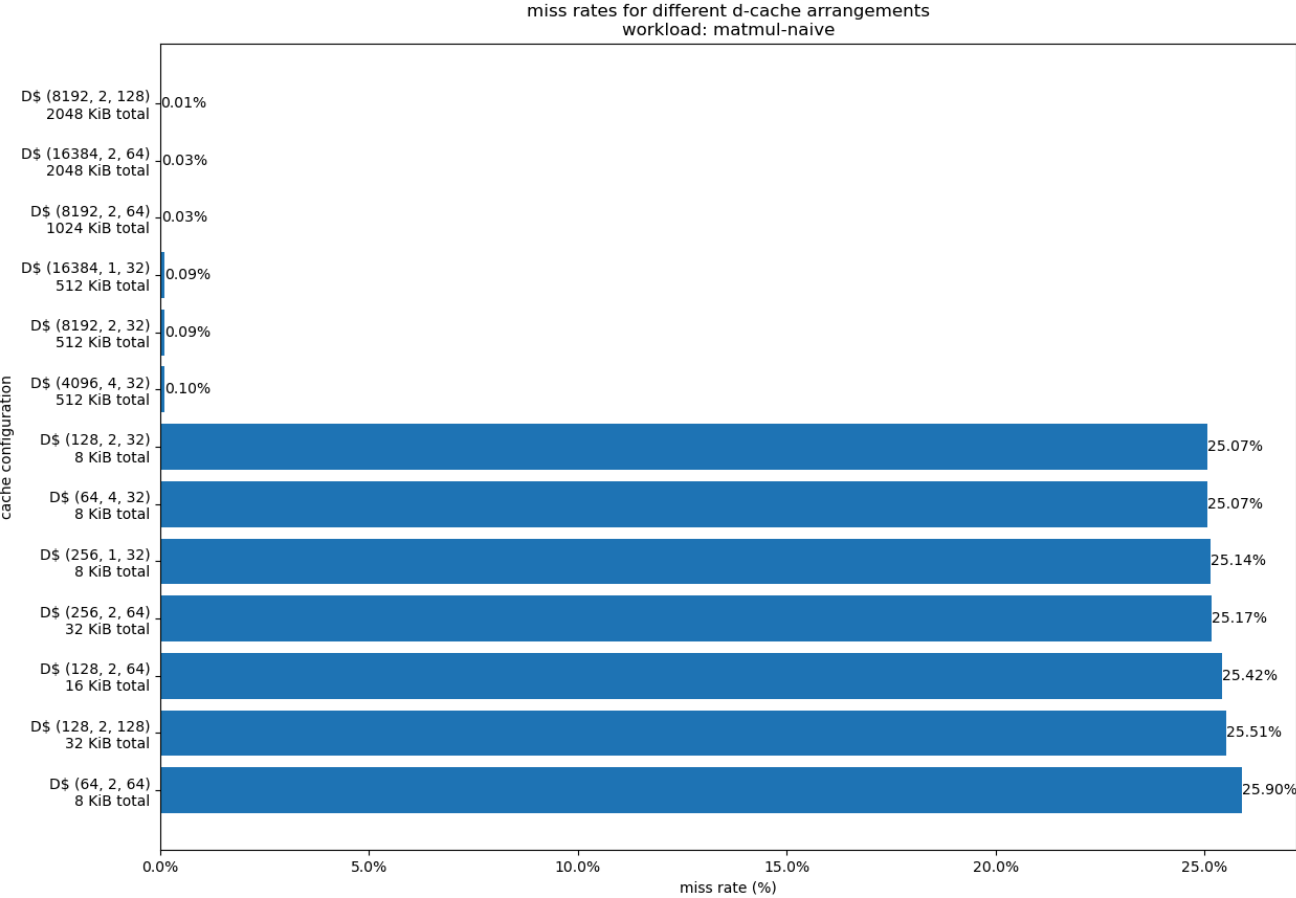
- No i-cache (small code size)
- No L2 cache (for simplicity)
- $M = N = 256$ (512 took way too long to run)

sets	ways	bsize	total size
256	1	32	8 KiB
128	2	32	8 KiB
64	4	32	8 KiB
64	2	64	8 KiB
128	2	64	16 KiB
256	2	64	32 KiB
128	2	128	32 KiB
16384	1	32	512 KiB
8192	2	32	512 KiB
4096	4	32	512 KiB
8192	2	64	1024 KiB
16384	2	64	2048 KiB
8192	2	128	2048 KiB

Table 1: Cache dimensions used in the experiment.







- Based on the results: choice of a 64 KiB i-cache (256:4:64)
- Counting instructions:
 - `riscv64-unknown-elf-objdump --disassemble=<symbol-name>`
 - `spike -g ...`
 - Parse histogram, sum counts from addresses within the function

- Implementation of vectorized `matmul`
 - GCC 14 and 15 do not yet support the Q extension
 - `riscv64-unknown-elf-as` does though
 - Workaround: manually implement `asm stub`, export it as symbol, call it from regular code
- Several hurdles:
 - `pk` needs to be compiled with support for the F/D/Q extensions
 - Need to run `spike` with the `--isa=rv64gqc` flag (otherwise `pk` falls back to softfloat and traps floating point instructions)
 - Need to take care with 128-bit alignment lest we get faults (`-00` makes the program crash due to this)

```
.global fmaddq_stub
```

```
fmaddq_stub:
```

```
    flq fa0, 0(a1)
```

```
    flq fa1, 0(a2)
```

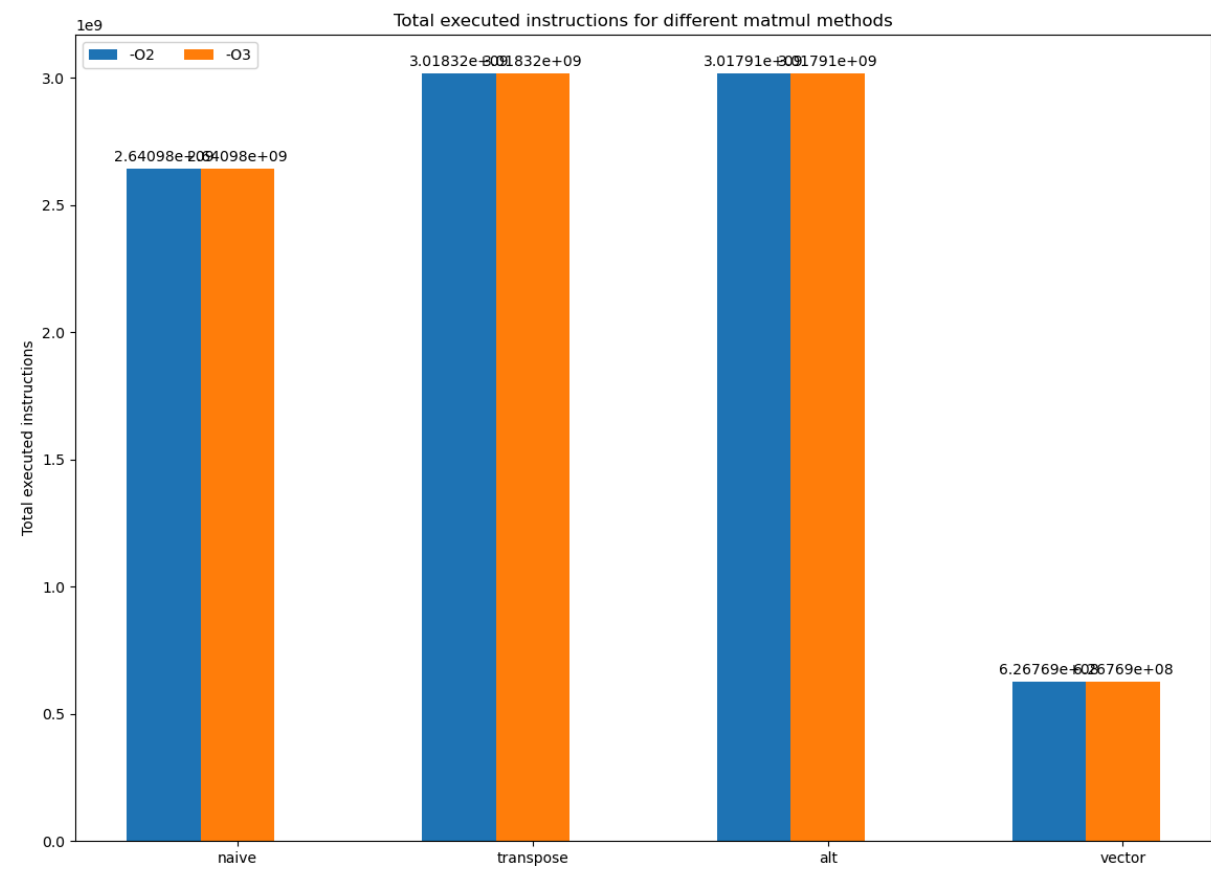
```
    flq fa2, 0(a3)
```

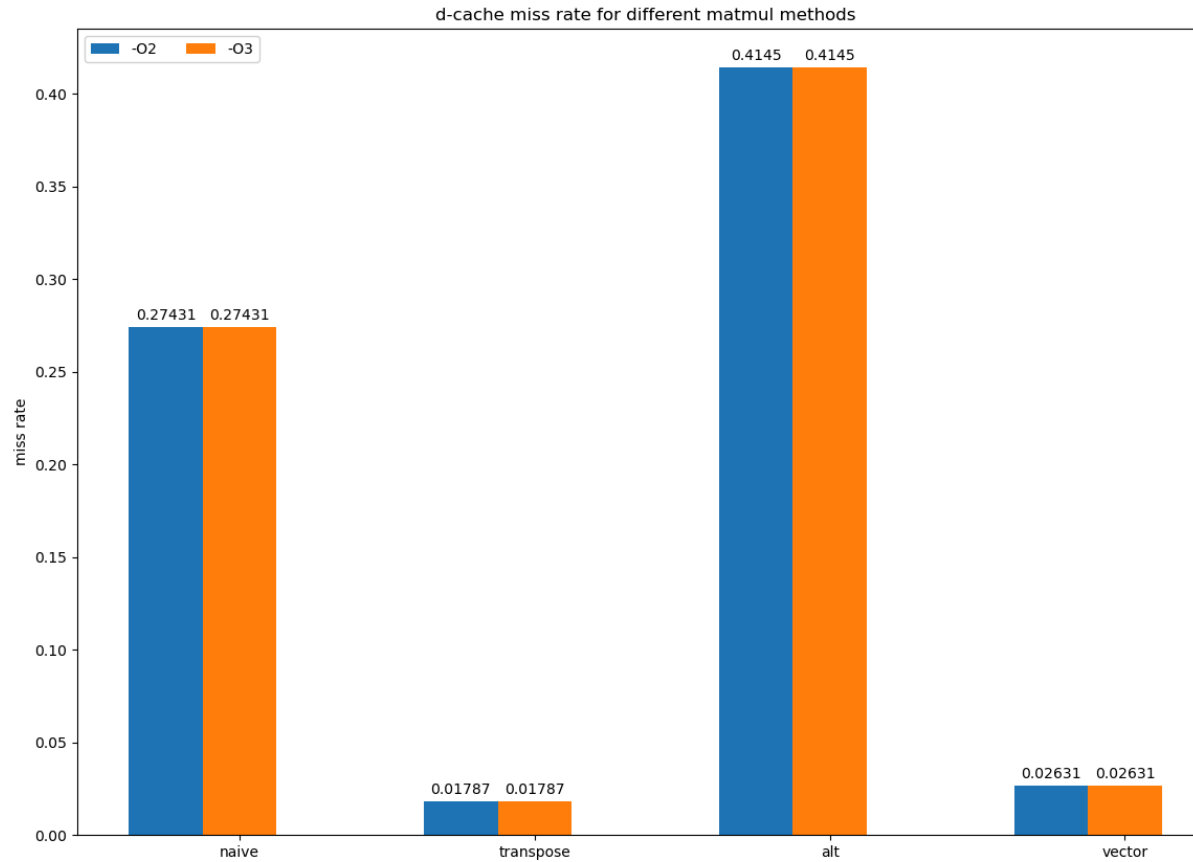
```
    fmadd.q fa0, fa0, fa1, fa2
```

```
    fsq fa0, 0(a0)
```

```
    ret
```

```
void fmaddq_stub(float result[4], float a[4], float b[4],  
float c[4]);
```



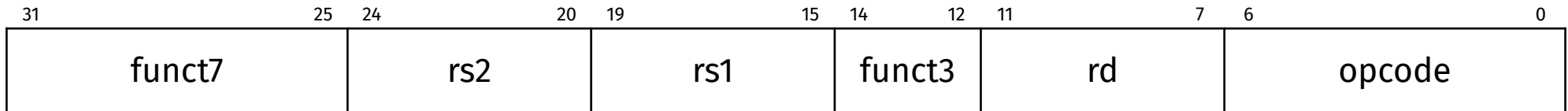


Takeaways

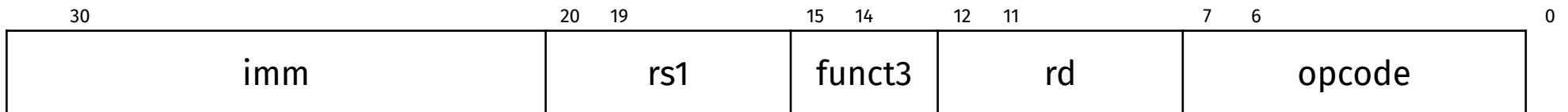
- naive and alt perform pretty bad cache locality-wise, transpose already much better
- The vectorized variant reduces the instruction count by 80% (very noticeable speedup in execution on Spike)
 - Seems to be worth using even at higher cost in latency/power consumption
- No significant improvement from -02 to -03 (but definitely from -00 to 02)

Lab 02: QEMU

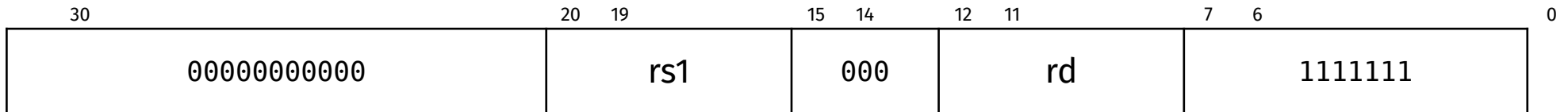
- R-type:



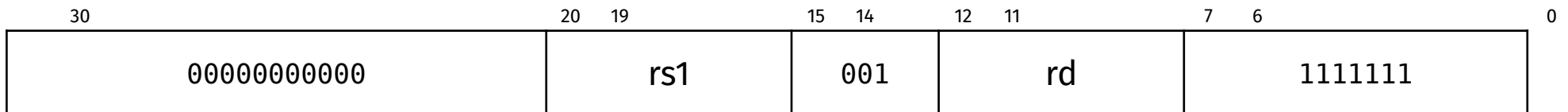
- I-type:



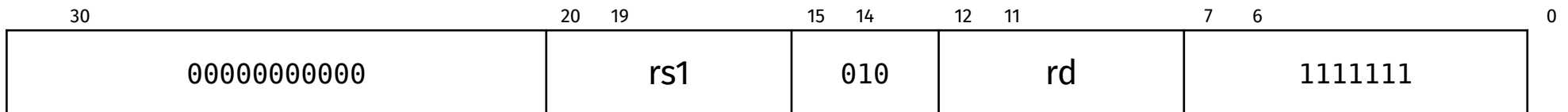
- bitrev:



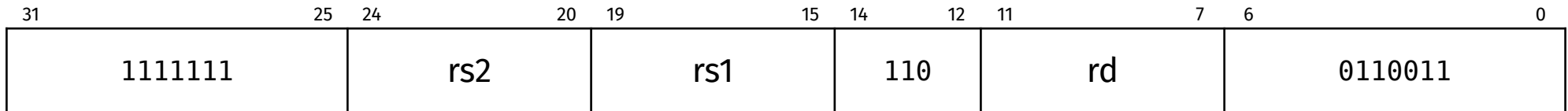
- cclz:



- cctz:



- multmod:



```
static bool trans_multmod(DisasContext *ctx, arg_multmod *a)
{
    TCGv rs1, rs2, rd;

    rs1 = get_gpr(ctx, a->rs1, EXT_NONE);
    rs2 = get_gpr(ctx, a->rs2, EXT_NONE);
    rd = get_gpr(ctx, a->rd, EXT_NONE);

    tcg_gen_mul_tl(rd, rd, rs1);
    tcg_gen_rem_tl(rd, rd, rs2);

    gen_set_gpr(ctx, a->rd, rd);

    return true;
}
```

Lab 03: gem5

- gem5 is a functional, event driven simulator for computer systems
- capable of emulating several arches (x86, arm64, riscv, ...)
- modular, composable, extensible

- Decided to build locally since I run Ubuntu 25.04 on my machine; ran into several build issues that I fixed along the way
- Was able to run through the best part of the “Learning gem5” tutorial
 - https://www.gem5.org/documentation/learning_gem5/introduction/
- Was able to set up and reproduce the tests in the SSCAD 2024 repo from LSC
- Struggling to grasp the details but understood the general workflow for extending the ISA and configuring uarch details within gem5

Final project proposal

- New extension currently under development: Supervisor Domain Access Protection
- <https://github.com/riscv/riscv-smmtt>
- Aims to enable confidential computing use cases on RISC-V
- Defines a data structure called MPT (Memory Protection Table) that configures and enforces read/write permissions to physical memory addresses (or device-mapped regions) based on SDIDs (Supervisor Domain Identifiers)

- **Proposal:** implement at least a subset of the following extensions in Spike
 - Smsdid: interface to program the active supervisor domain under which a hart is operating
 - Smmpt: extension to set the access permissions for a memory region or page associated with a supervisor domain
- **Objective:** develop a proof-of-concept of the extensions, showing effective resource isolation between different SDIDs